Lab Report — Escaneo de Puertos y Filtrado con Firewall (GCP)

- **Objetivo:** practicar fundamentos de seguridad de redes — descubrimiento de servicios con escaneo de puertos (nmap) y control de exposición con reglas de firewall (VPC) — en un entorno de laboratorio aislado.
- **Entorno:** Google Cloud Platform. VM `lab-firewall` (Compute Engine `e2-micro`, Debian 12, zona `us-central1-a`, etiqueta `lab-web`). Herramienta de escaneo: **nmap 7.94** desde Cloud Shell.
- **Alcance:** solo la VM de laboratorio propia. Recurso efímero, eliminado al finalizar.

Metodología y resultados

1. Despliegue de la VM

```
gcloud compute instances create lab-firewall --zone=us-central1-a \
  --machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud --tags=lab-web
```

IP externa asignada para las pruebas.

2. Reconocimiento (escaneo completo)

```
sudo nmap -sS -sV -sC -p- -T4 -oN recon-inicial.txt <IP>
```

Resultado: de 65.535 puertos, **un solo puerto abierto**.

```
Not shown: 65433 filtered tcp ports (no-response), 100 filtered (port-unreach)
22/tcp   open   ssh   OpenSSH 9.2p1 Debian 2+deb12u10
3389/tcp closed ms-wbt-server
```

Filtrado del resultado a solo lo abierto:

```
grep open recon-inicial.txt   # 22/tcp open ssh ...
```

3. Análisis de los tres estados observados

Puerto	Estado	Interpretación
22 (SSH)	`open`	Servicio SSH escuchando **+** regla `default-allow-ssh` de GCP lo permite
3389 (RDP)	`closed`	Regla `default-allow-rdp` lo permite, pero **no hay servicio** (VM Linux)
80, 443, resto	`filtered`	**Sin regla** que los permita → el firewall descarta el paquete

4. Prueba de control: abrir un puerto con el firewall

```
gcloud compute firewall-rules create lab-allow-http \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --target-tags=lab-web --source-ranges=0.0.0.0/0

nmap -Pn -p 80 <IP>   # Resultado: 80/tcp closed (antes filtered)
```

El cambio **`filtered`** → **`closed`** confirma que la regla de firewall funcionó: el paquete ahora **alcanza la VM** (aunque no haya servicio escuchando).

5. Cierre y limpieza

```
gcloud compute firewall-rules delete lab-allow-http     # cerrar puerto = borrar la regla
gcloud compute instances delete lab-firewall --zone=us-central1-a   # evitar costos
```

Lecciones y conceptos aplicados

- **Estados de puerto:** `open` (servicio + permitido), `closed` (permitido sin servicio), `filtered` (bloqueado por firewall). La distinción `closed` vs `filtered` revela si existe una regla que permite el tráfico.
- **Firewall = control de exposición:** crear regla `ALLOW` = **abrir**; borrar la regla = **cerrar**. Aplicar mínima exposición (least exposure) cerrando lo que no se usa.
- **nmap solo observa:** el escáner no modifica el objetivo; quien abre/cierra es el firewall.
- **SYN scan (`-sS`):** técnica estándar (half-open), rápida y sigilosa; requiere privilegios.
- **Reconocimiento vs verificación:** primera vez = amplio y profundo (`-p-` , `-sV` , `-sC` , guardar salida); tras un cambio = puntual (`-Pn -p PUERTO`).

Hallazgo de hardening

La VM exponía **SSH (22) a `0.0.0.0/0`** (toda internet) por la regla por defecto — superficie innecesaria vulnerable a fuerza bruta. **Remediación recomendada:** restringir el origen a la IP del administrador (`--source-ranges=TU_IP/32`) o usar un bastión/IAP.

Relevancia y nota sobre arquitecturas serverless

Estos conceptos aplican a servidores/VMs tradicionales. En arquitecturas **serverless** (como el sitio de producción en Firebase) **no hay puertos que administrar**: el control de acceso equivalente se implementa con **reglas de Firestore/Storage, App Check y RBAC** (documentados aparte). Este lab consolida los fundamentos de red transferibles a cualquier infraestructura tradicional.